

DEADLOCKS

Mata Kuliah Sistem Operasi

Dosen Pengampu: Triawan Adi Cahyanto, M.Kom

Program Studi: Teknik Informatika

Universitas: Muhammadiyah Jember

Process(P1)

Process(P2)

Allocated

Requesting

Requesting

Allocated

Daftar Isi

- 1 Deadlocks 
- 2 System Model 
- 3 Deadlock in Multithreaded Applications 
- 4 Deadlock Characterization 
- 5 Methods for Handling Deadlocks 
- 6 Deadlock Prevention 
- 7 Deadlock Avoidance 
- 8 Deadlock Detection 
- 9 Recovery from Deadlock 
- 10 Kesimpulan 

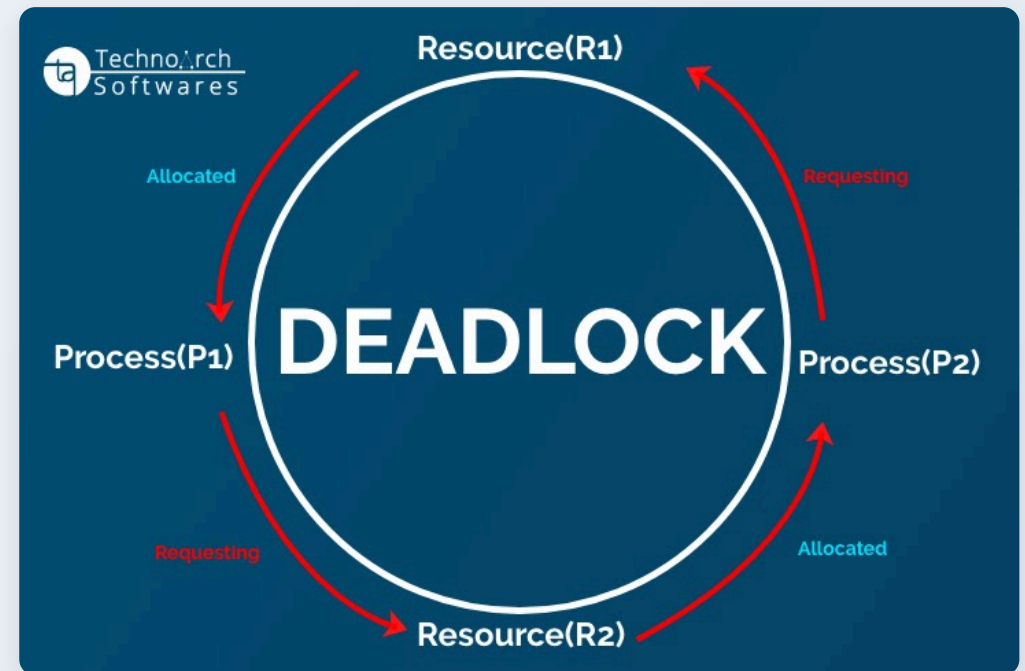
Deadlocks

Definisi Deadlock

Situasi di mana sekumpulan proses macet secara **permanen** karena setiap proses menunggu sumber daya yang dipegang oleh proses lain.

Contoh Deadlock

- Proses P1 memegang sumber daya R1
- Proses P2 memegang sumber daya R2
- P1 menunggu R2 yang dipegang P2
- P2 menunggu R1 yang dipegang P1
- **Kedua proses macet selamanya**



System Model

Konsep Dasar

Sistem terdiri dari sumber daya **terbatas** yang didistribusikan ke proses-proses yang **bersaing**

Jenis Sumber Daya



Reusable: dapat digunakan kembali



Consumable: habis saat digunakan



Preemptable: dapat diambil



Nonpreemptable: tidak dapat diambil

Alur Penggunaan Sumber Daya



Request



Use



Release

Kapan Deadlock Terjadi?

Deadlock umumnya terjadi pada sumber daya:

- **Reusable** - dapat digunakan kembali
- **Nonpreemptable** - tidak dapat diambil

Contoh: printer, file, database, semaphore

Deadlock in Multithreaded Applications

Definisi Deadlock Multithreading

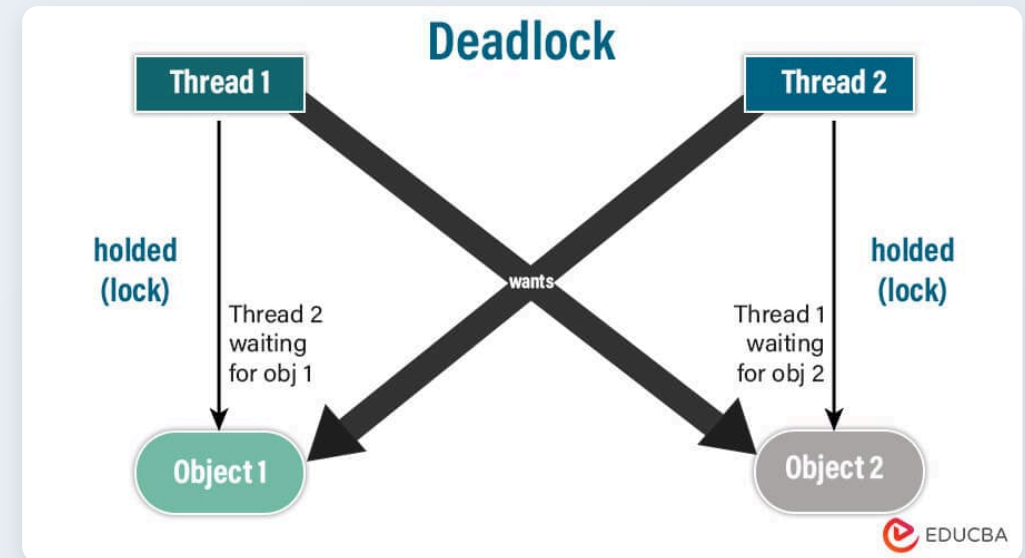
Situasi di mana **dua atau lebih thread** diblokir selamanya, masing-masing menunggu thread lain untuk melepaskan sumber daya.



Aplikasi multithreaded **rentan deadlock** karena beberapa thread bersaing untuk sumber daya bersama

Contoh Deadlock

- Thread 1 memegang lock Object 1
- Thread 2 memegang lock Object 2
- Thread 1 menunggu lock Object 2
- Thread 2 menunggu lock Object 1
- **Kedua thread macet selamanya**



Deadlock Characterization

❌ Mutual Exclusion

Hanya **satu proses/thread** dapat menggunakan sumber daya pada waktu tertentu

⌚ Hold and Wait

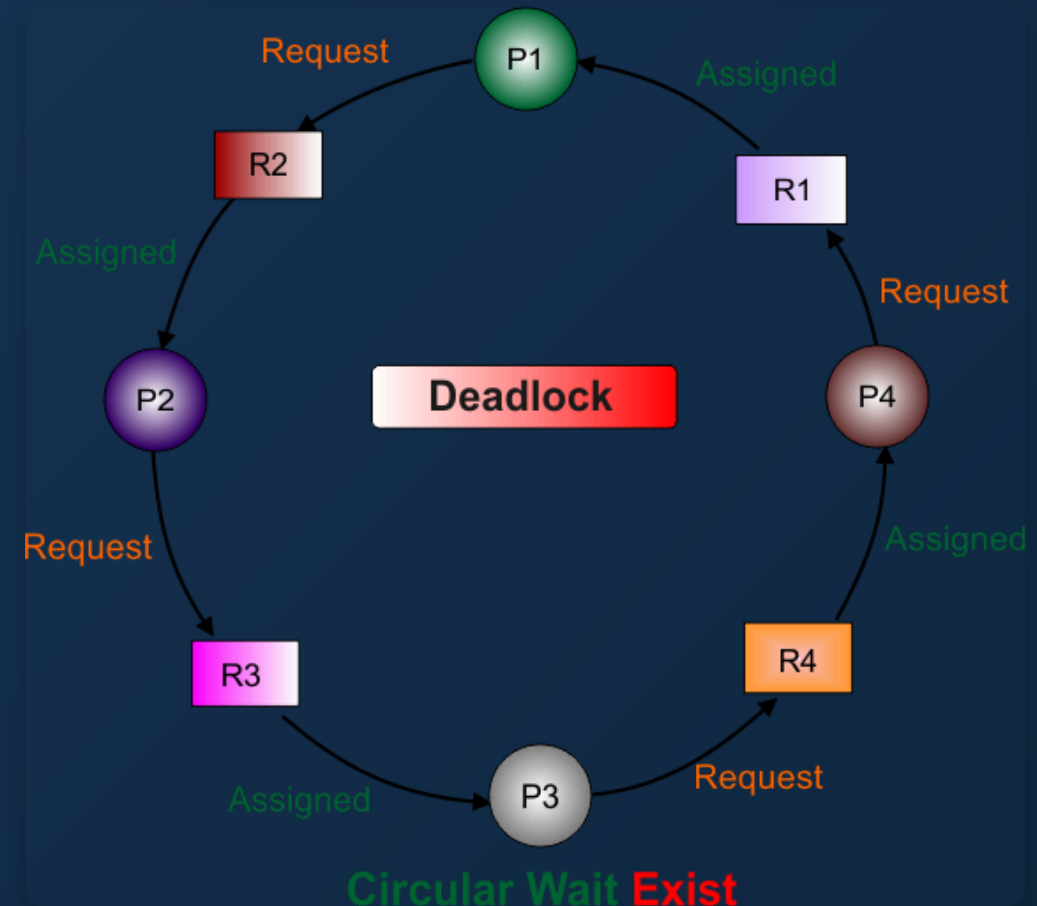
Proses/thread **memegang sumber daya** sambil menunggu sumber daya lain

🚫 No Preemption

Sumber daya **tidak dapat diambil** dari proses/thread kecuali dilepaskan

🔄 Circular Wait

Sekumpulan proses/thread **menunggu satu sama lain** secara sirkular



💡 Penting

Keempat kondisi ini **harus terjadi bersamaan** untuk deadlock terjadi. Memecahkan salah satu kondisi dapat mencegah deadlock.

Methods for Handling Deadlocks

Ada empat pendekatan utama untuk menangani deadlock dalam sistem operasi:



Deadlock Prevention

Memastikan **salah satu kondisi deadlock** tidak terjadi dengan:

- Menghilangkan mutual exclusion
- Mencegah hold and wait
- Mengizinkan preemption
- Menghindari circular wait



Deadlock Avoidance

Membuat keputusan **dinamis** untuk mencegah deadlock:

- Banker's Algorithm
- Resource Allocation Graph
- Memeriksa keamanan sistem
- Menolak permintaan berisiko



Deadlock Detection

Mendeteksi deadlock yang telah terjadi:

- Wait-for Graph
- Resource Allocation Graph
- Thread Dump Analysis
- Pemeriksaan berkala



Recovery from Deadlock

Memulihkan sistem dari deadlock yang terdeteksi:

- Process Termination
- Resource Preemption
- Rollback
- Reboot sistem (terakhir)

Deadlock Prevention

Mencegah deadlock dengan memastikan **salah satu dari empat kondisi deadlock tidak terjadi**:



Mencegah Mutual Exclusion

- Membuat sumber daya **dapat dibagi (sharable)**
- Menggunakan sumber daya virtual
- Contoh: printer spooling, file read-only



Mencegah Hold and Wait

- Proses meminta **semua sumber daya sekaligus**
- Atau melepaskan sumber daya sebelum meminta yang baru
- Contoh: atomic allocation



Mencegah No Preemption

- Mengizinkan **preemption** (pengambilan paksa)
- OS dapat mengambil sumber daya dari proses
- Contoh: memory management



Mencegah Circular Wait

- Memberikan **urutan numerik** pada sumber daya
- Proses harus meminta sumber daya sesuai urutan
- Contoh: lock ordering dalam multithreading



Prevention lebih efektif daripada deteksi, namun sering kali **mengurangi efisiensi sistem** karena pembatasan akses sumber daya

Deadlock Avoidance

Menghindari deadlock dengan membuat **keputusan dinamis** untuk memastikan sistem tidak masuk ke kondisi tidak aman



Safe State

- Sistem dalam keadaan **aman** jika ada urutan eksekusi proses yang memungkinkan semua proses selesai
- OS dapat mengalokasikan sumber daya tanpa risiko deadlock
- Sistem dapat memenuhi permintaan proses saat ini dan di masa depan



Unsafe State

- Sistem dalam keadaan **tidak aman** jika tidak ada urutan eksekusi proses yang memungkinkan semua proses selesai
- Memungkinkan terjadinya deadlock
- OS harus menolak permintaan sumber daya yang menyebabkan keadaan tidak aman



Banker's Algorithm

- Digunakan ketika ada **beberapa instance** dari sumber daya
- Memeriksa keamanan sistem sebelum mengalokasikan sumber daya
- Mensimulasikan alokasi untuk memastikan sistem tetap aman
- Menolak permintaan yang menyebabkan keadaan tidak aman



Resource Allocation Graph

- Digunakan ketika setiap sumber daya hanya memiliki **satu instance**
- Memeriksa siklus dalam graf untuk menghindari keadaan tidak aman
- Proses ditandai dengan lingkaran, sumber daya dengan persegi
- Alokasi ditandai dengan panah dari sumber daya ke proses

💡 Avoidance memerlukan **informasi lengkap** tentang sumber daya yang dibutuhkan proses di masa depan, yang sering kali sulit diperoleh

Deadlock Detection

Mendeteksi deadlock yang telah terjadi untuk dapat **menanganinya dengan tepat**



Kapan Mendeteksi

- **Berkala** - pemeriksaan rutin sistem
- Saat proses **menunggu terlalu lama**
- Saat ada permintaan sumber daya baru
- Dipicu oleh event tertentu dalam sistem



Wait-for Graph

- Struktur graf untuk **mendeteksi siklus**
- Node = proses/thread
- Edge = proses menunggu sumber daya
- Siklus dalam graf menunjukkan deadlock



Teknik Deteksi

- **Thread Dump Analysis** - analisis status thread dan lock
- **Static Analysis** - review kode untuk circular waits
- **Runtime Monitoring** - peringatan pada lock lama
- **Resource Allocation Graph** - deteksi siklus



Keuntungan Deteksi

- Memungkinkan akses sumber daya **tanpa batasan**
- Pemenuhan permintaan sumber daya **segera**
- Lebih fleksibel dibandingkan prevention
- Dapat menangani deadlock yang kompleks



Deteksi deadlock memerlukan **overhead komputasi** yang signifikan, namun memungkinkan sistem beroperasi lebih efisien

Recovery from Deadlock

Memulihkan sistem dari deadlock yang telah **terdeteksi** untuk memulihkan operasi normal



Process Termination

- Menghentikan **satu atau lebih proses** deadlock
- Abort semua proses deadlock
- Atau hentikan proses satu per satu hingga deadlock terpecahkan
- Pertimbangan: biaya, prioritas, sisa eksekusi



Resource Preemption

- Ambil **sumber daya** dari proses
- Alokasikan kembali ke proses lain
- Pemilihan korban berdasarkan:
 - Prioritas proses
 - Sumber daya yang dimiliki
 - Sumber daya yang dibutuhkan



Rollback

- Kembalikan proses ke keadaan **sebelum deadlock**
- Menggunakan checkpoint untuk menyimpan status
- Membutuhkan sistem yang mendukung rollback
- Bisa mengulang eksekusi dari checkpoint terakhir



Deadlock Ignorance

- **Mengabaikan deadlock** sama sekali
- Disebut **Ostrich Algorithm**
- Hanya berlaku jika deadlock sangat jarang terjadi
- Solusi: reboot sistem jika deadlock terjadi



Recovery dari deadlock sering kali **memerlukan pengorbanan** seperti kehilangan data atau proses yang tidak selesai

Kesimpulan



Deadlock adalah **masalah serius** dalam sistem operasi yang dapat menyebabkan sistem macet



Deadlock terjadi ketika **empat kondisi** terpenuhi: Mutual Exclusion, Hold and Wait, No Preemption, dan Circular Wait



Ada **empat metode** penanganan deadlock: Prevention, Avoidance, Detection, dan Recovery



Pemilihan metode penanganan deadlock tergantung pada **karakteristik sistem** dan kebutuhan aplikasi



Pemahaman tentang deadlock sangat **penting untuk programmer** yang mengembangkan aplikasi multithreaded